

Foreach Loop in C# with Examples

In this article, I am going to discuss **Foreach Loop in C#** with Examples. Please read our previous article where we discussed [Generic Queue in C#](#) with Examples.

What is Foreach Loop in C#?

Looping is a way to execute a statement(s) multiple times depending on the result of a condition. As long as the given condition satisfies, the loop executes. Once the condition fails, the loop terminates.

The Foreach Loop in C# is a different kind of loop that doesn't include initialization, termination, and increment/decrement characteristics. It uses the collection to take values one by one and then processes them.

The foreach loop in C# is used to iterate over the elements of a collection. Here, the collection may be an array or a list or a dictionary, etc. As per the name i.e. foreach, it executes the loop body for each element present in the array or collection.

In C#, the foreach loop iterates collection types such as Array, ArrayList, List, Hashtable, Dictionary, etc. It can be used with any type that implements the IEnumerable interface.

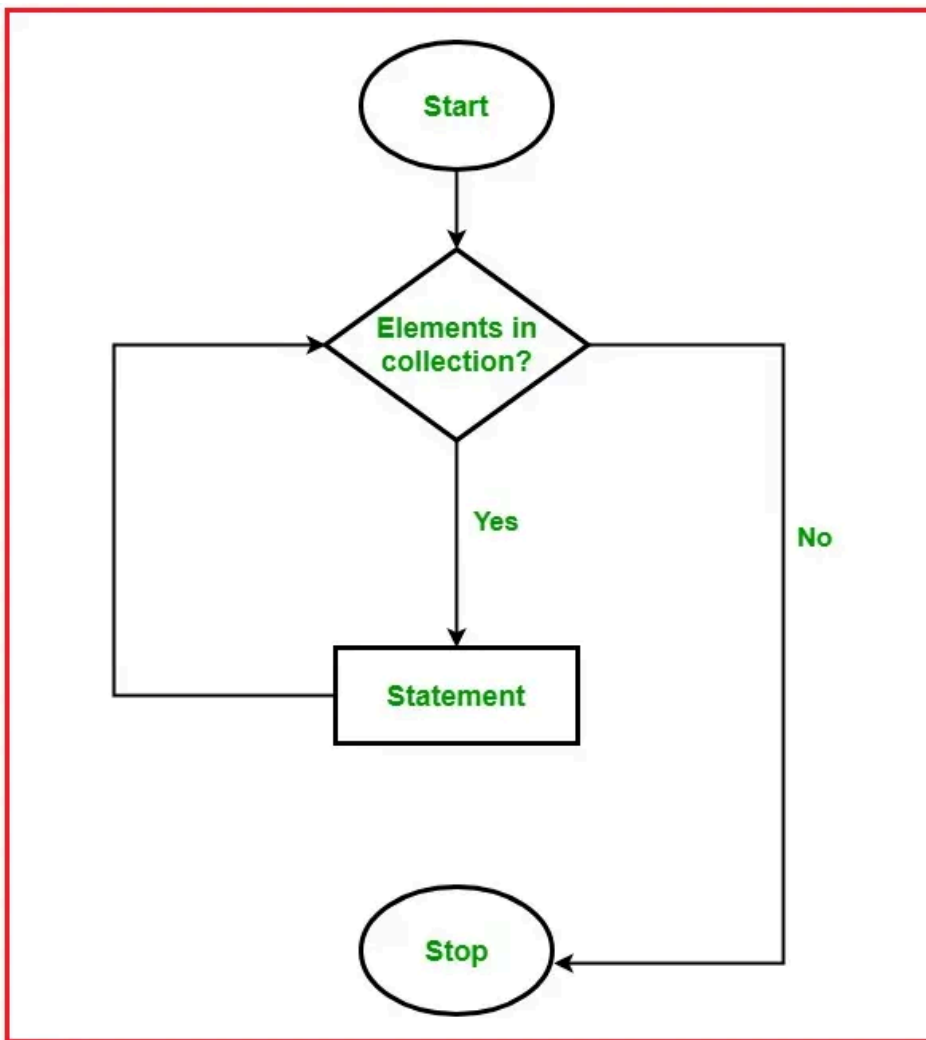
Syntax to use Foreach Loop in C# Language:

The System.Collections.Generic namespace contains the `ForEach()` extension method that can be used with any built-in collection classes such as List, Dictionary, SortedList, etc.

```
foreach(data_type var_name in collection_variable)
{
    // statements to be executed
}
```

As shown in the above syntax, it is necessary to enclose the statements of the foreach loop in curly braces `{}`. Here, instead of declaring and initializing a loop counter variable, we need to declare a variable of the same type as the collection type or array type, followed by the `in` keyword, which is then followed by the array name or collection name. In the foreach loop body, you can use the loop variable you created rather than using an indexed array element or the index position of the collection.

Flowchart to Understand Foreach Loop:



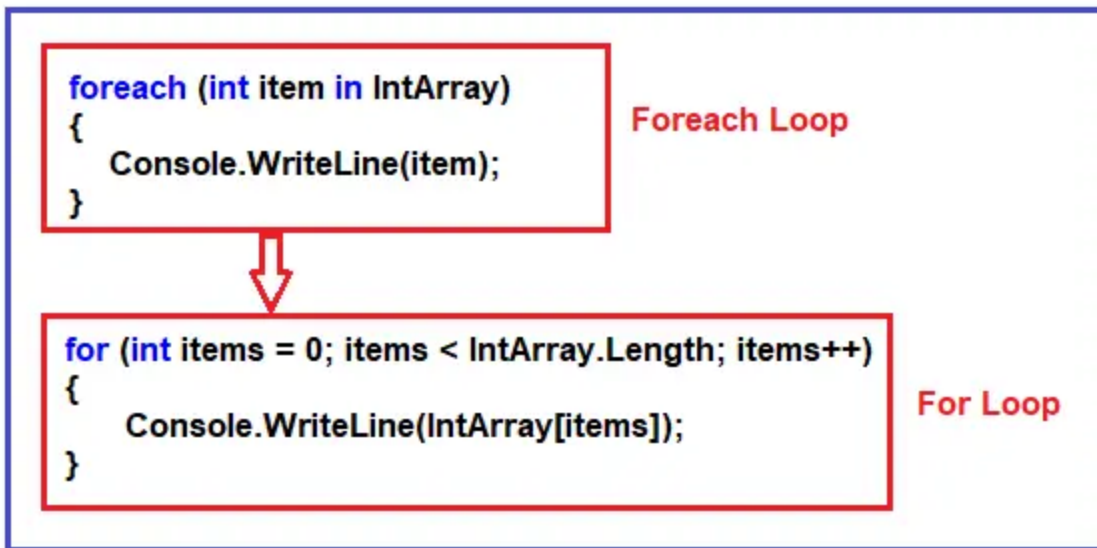
Example to understand the use of foreach loop in C#:

```
using System;
namespace ForeachLoopDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // creating an array of integer type
            int[] IntArray = new int[] { 1, 2, 3, 4, 5, 6, 7 };
            Console.WriteLine("Print Array Elements using Foreach Loop:");
            // The foreach loop will run till the last element of the array
            foreach (int item in IntArray)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Print Array Elements using Foreach Loop:
1
2
3
4
5
6
7
```

The following image shows the equivalent for loop of the above foreach loop in C# Language.



Note: The keyword "in" in the foreach loop is used to iterate over the collection. The in keyword selects an item from the collection on each iteration and stores it in the variable element. On the first iteration, the first item of the collection is stored in element. On the second iteration, the second element is selected, and so on. The number of times the foreach loop will execute is equal to the number of elements in the array or collection.

Example to understand Foreach loop in C# Language

```
using System;
namespace ForeachLoopDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] Marks = { 115, 135, 105, 116, 122 };
            Console.Write("Marks :");
            foreach (int mark in Marks)
            {
                Console.Write($" {mark}");
            }
        }
    }
}
```

```

    }
    Console.WriteLine();
    int Highest_Mark = Maximum(Marks);
    Console.WriteLine("The Highest Mark is " + Highest_Mark);
    Console.ReadKey();
}
public static int Maximum(int[] Marks)
{
    int maxSoFar = Marks[0];
    // for each loop
    foreach (int mark in Marks)
    {
        if (mark > maxSoFar)
        {
            maxSoFar = mark;
        }
    }
    return maxSoFar;
}
}
}

```

Output:

```

Marks : 115 135 105 116 122
The Highest Mark is 135

```

Example: Foreach Loop with Dictionary Collection in C#

The following example shows how to use foreach loop on a dictionary collection using C# language. The Dictionary collection class belongs to System.Collections.Generic namespace.

```

using System;
using System.Collections.Generic;
namespace ForeachLoopDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var citiesOfCountry = new Dictionary<string, string>()
            {
                {"UK", "London, Birmingham"},
                {"USA", "Chicago, Washington"},
                {"India", "Mumbai, Pune"}
            };
            foreach (var city in citiesOfCountry)
            {
                Console.WriteLine("{0}, {1}", city.Key, city.Value);
            }
        }
    }
}

```

```

    }
    Console.ReadKey();
}
}
}

```

Output:

```

UK, London, Birmingham
USA, Chicago, Washington
India, Mumbai, Pune

```

Implement IEnumerable Interface in C#

As mentioned earlier, the foreach loop in C# can be used to iterate any class that implements the IEnumerable interface. The following example shows how to implement the IEnumerable interface in order to use the foreach loop with the custom class.

```

using System;
using System.Collections;
namespace ForeachLoopDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Shop objShop = new Shop();
            foreach (Customer cust in objShop)
            {
                Console.WriteLine(cust.Name);
            }
            Console.ReadKey();
        }
    }
    class Customer
    {
        public int Id { get; set; }
        public string Name { get; set; }
    }
    class Shop : IEnumerable
    {
        private Customer[] CustomerArray = new Customer[4];
        public Shop()
        {
            CustomerArray[0] = new Customer() { Id = 1, Name = "James" };
            CustomerArray[1] = new Customer() { Id = 2, Name = "Smith" };
            CustomerArray[2] = new Customer() { Id = 3, Name = "Sara" };
            CustomerArray[3] = new Customer() { Id = 4, Name = "Pam" };
        }
    }
}

```

```

//Here implementation for the GetEnumerator method.
public IEnumerator GetEnumerator()
{
    return CustomerArray.GetEnumerator();
}
}
}

```

Output:

```

James
Smith
Sara
Pam

```

In the above example, the Shop class has implemented the IEnumerable interface that contains the GetEnumerator() method. This will enable the Shop class to be used with the foreach loop that returns the Customer objects.

Foreach with List Collection in C#

In the below example, we use foreach loop to iterate over a collection of List type

```

using System;
using System.Collections.Generic;
namespace ForeachLoopDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<string> Countries = new List<string> { "India", "USA", "UK",
            foreach (string country in Countries)
            {
                Console.WriteLine(country);
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```

India
USA
UK
Canada

```

Limitations of Foreach Loop:

1. The Foreach loop in C# is not appropriate when we want to modify the array or collection.

```
foreach(int item in collection)
{
    // only changes item variable not the collection element
    item = item + 2;
}
```

2. The Foreach loop in C# does not keep track of indexes. So, it is not possible to get the array index or collection index using the Foreach loop

```
foreach (int item in collection)
{
    if item == target)
    {
        return ???; // do not know the index of the item
    }
}
```

3. The Foreach loop in C# only iterates forward direction. The following forloop cannot be converted to a foreach loop as we are starting from the last element

```
for (int i = numbers.Length - 1; i > 0; i--)
{
    Console.WriteLine(numbers[i]);
}
```

Difference between For Loop and Foreach Loop:

The for loop in C# executes a statement or a block of statements as long as the given condition is true. Whereas foreach loop executes a statement or a block of statements for each element present in the array or collection and there is no need to define the minimum or maximum limit.

In for loop, it is possible to iterate the array elements in both forward and backward directions, e.g from index 0 to 9 and from index 9 to 0. But in the foreach loop, it is only possible to iterate an array in the forward direction, backward direction is not possible.

Points to Remember while working with Foreach Loop:

1. The foreach loop in C# iterates only in the forward direction. Backward direction is not possible.
2. From the Performance point of view, the foreach loop takes more time as compared with for loop. Because internally it uses extra memory space.
3. The foreach loop use GetEnumerator() method of the IEnumerable interface. So, the foreach loop can be used with any class that has implemented the interface.
4. We Can exit the foreach loop in C# by using break, return, Goto and throw statements.

In the next article, I am going to discuss the [Generic HashSet<T> Collection Class in C#](#) with Examples. Here, in this article, I try to explain **Foreach Loop in C#** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.